

Lecture 3 Divide and Conquer -- sorting and max subarray

Sorting

Why Sorting

- Problem become easy once items are in sorted order(find a median, binary search)

Insertion Sort

```
Insert-sort(A,n)
  for j <- 2 to n
    insert key A[j] into the already sorted subarray A[1,...,j-1]
    by pairwise key-swaps down to its right position
```

$\Theta(n^2)$ time complexity by $\Theta(n^2)$ comparisons and $\Theta(n^2)$ swaps

Binary Insertion Sort

```
Binary-insertion-sort(A,n)
  for j <- 2 to n
    insert key A[j] into the already sorted subarray
    but use binary search to find the right position
```

$\Theta(n \log n)$ comparisons and $\Theta(n^2)$ swaps. (Still $\Theta(n^2)$)

Merge Sort

```
Merge-sort A[1...n]
  If n = 1, done
  Recursively sort A[1...n/2] and A[n/2+1 ... n]
  merge two sorted list
```

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n).$$

By master theorem, $\log_b a = 1, d = 1$. Thus the complexity is $O(n \log n)$

Priority Queue

A data structure implementing a set S of elements, each associated with a key, supporting the following operations

- insert
- max
- extract_max
- increase_key

Heap

Implementation of a priority queue. Implemented by complete binary tree.

The key of a node $\geq$ the keys of its children for max heap property